

DARS: Making Problem Difficulty Simple, Fair, and Interpretable for Online Coding Assessment

Khethavath Sunil Naik

BTech Project, India

Abstract—Online coding platforms like LeetCode are flooded with data, but turning that data into useful difficulty predictions remains surprisingly hard. Most systems either use fixed labels, apply opaque neural models, or ignore problem relationships. We introduce DARS—a practical system that combines classical rating theory, problem metadata, and graph-aware routing. The key insight is that a simple logistic regression model trained on carefully engineered features can actually outperform complex deep learning approaches while remaining fully auditable. We test this idea on 1,825 LeetCode problems and compare against Elo, Glicko-2, Bayesian Knowledge Tracing, Deep Knowledge Tracing, Self-Attentive Knowledge Tracing, and Graph Neural Knowledge Tracing. With no target leakage, DARS achieves 76.71% accuracy and 0.0375 calibration error—better than all baselines. We also expose where the system breaks: cold-start problems (new or unpopular items), fairness gaps between difficulty groups, and the fact that offline prediction metrics alone tell us nothing about whether students actually learn. This paper is meant to be practical: if you’re building an adaptive system, we show what works, what doesn’t, and why simplicity and interpretability are not the enemy of accuracy.

I. INTRODUCTION

A. The Real Problem

Imagine you’re building an online coding platform. Every day, learners attempt thousands of problems. You can see which problems they solve and which ones stump them. You have metadata: acceptance rates, discussion counts, community ratings, topic tags. The natural question is: can you predict whether a problem is Easy, Medium, or Hard? And once you can, can you use those predictions to recommend the next problem that is just challenging enough?

The honest answer today is messy. You could hard-code the labels (what LeetCode does), but those labels are inconsistent and stale. You could train a neural network (what researchers do), but then no one can audit why a problem is marked Hard. You could apply classical item-response theory, but it assumes the problem difficulty is fixed and unchanging. What you really want is something simple enough to trust, accurate enough to deploy, and flexible enough to improve over time.

This paper describes DARS: Dynamic Adaptive Rating System. It is not revolutionary. It combines ideas from educational testing (Elo ratings), feature engineering (composite metadata signals), and graph theory (problem relationships). But the combination works surprisingly well. On real LeetCode data with 1,825 problems, a simple logistic regression model with

11 engineered features reaches 76.71% accuracy at binary difficulty classification—beating modern deep learning baselines like SAKT (73.42%) and DKT (71.23%). The probability estimates are also well-calibrated (ECE 0.0375), which matters when you need to decide whether to push a learner or back off.

The catch? The system is only as good as the data you feed it. Cold-start problems (brand new, few solves) are nearly impossible to predict (62.64% accuracy vs. 84.62% for established problems). And we have a fairness problem: the system is much better at recalling Hard problems (94.07% sensitivity) than Easy ones (27.37%), which could lead to demotivating experiences for struggling learners.

We have also thought about what success actually looks like. Prediction accuracy is comforting but not the goal. The real goal is: did the learner master the intended skill? We do not have that data in a static LeetCode snapshot. So we outline what a live adaptive assessment experiment would need to measure—engagement, problem completion, skill progression, sequencing efficiency—to know whether DARS actually helps learners learn better.

B. What We Did

We took 1,825 LeetCode problems and extracted 20 metadata features each (acceptance rate, frequency percentile, community rating, likes, dislikes, discussion count, topics, related problems, etc.). We engineered 11 predictive features: normalized versions of the raw signals, plus composites like Difficulty Index (inverse acceptance), Reputation Score, Engagement Factor, and Complexity Score. We used logistic regression, Elo, Glicko-2, BKT, DKT, SAKT, and GNKT to predict binary difficulty on a stratified 80–20 split (1,460 training problems, 365 test).

On top of basic accuracy and AUC, we computed calibration (ECE), fairness metrics (TPR parity across groups), feature importance (ablation), robustness (noise perturbation), and category-wise performance. We also tested four graph construction strategies: skill-only (shared topics), temporal transition (co-occurrence order), expert prerequisite (domain knowledge), and hybrid (weighted combination). The expert prerequisite graph—basically a manual list of “learn this before that”—outperformed learned alternatives by 4.4 percentage points, which is a humbling result for machine learning.

C. Main Findings

- 1) **Simple beats Complex:** A logistic regression on engineered features outperforms SAKT, DKT, and GNKT. Class imbalance and limited training data favor interpretable models.
- 2) **Domain Knowledge Wins:** Expert-curated prerequisite graphs (72.88% accuracy) beat automatically constructed skill-only graphs (68.49%). You cannot always learn from data alone.
- 3) **Frequency is Everything:** A single feature—problem popularity—accounts for most predictive power. Removing it drops AUC from 0.7485 to 0.6792 (9% loss). Engagement patterns are actually counterintuitive: heavily discussed problems are easier (negative coefficient), which suggests they get discussed because beginners struggle with comprehension, not difficulty.
- 4) **Cold-Start is a Wall:** Predicting difficulty for low-acceptance problems (< 25% solve rate) is 22 percentage points harder than for established problems. This is not a model limitation; it is an information limitation.
- 5) **Fairness Matters:** The model is 66.7% biased toward Hard predictions due to class imbalance. This could be a feature (safety margin) or a bug (demotivation). You have to choose explicitly.
- 6) **Offline Metrics Lie:** A model that predicts difficulty does not prove that routing problems by predicted difficulty will help learners learn. We need experiments with actual learner outcomes.

II. WHAT OTHERS HAVE DONE

A. The Rating Game

The oldest approach is Elo, from chess. You give each player (or in our case, problem) a rating. When a player beats an opponent, their rating goes up. The update is dead simple: $R' = R + K(y - p)$, where K is how much credit you give for an upset. Elo works online, is easy to explain, and has been used in education (Khan Academy, some adaptive systems). But it only looks at win/loss. It ignores that some wins are harder than others.

Glicko-2 adds uncertainty. If a player has played many games, their rating is more stable. If they have not played in a while, their rating becomes less certain. This is clever, but it still only observes binary correctness.

B. Knowledge Tracing Models

Educational researchers tried a different angle: model what the learner knows. In Bayesian Knowledge Tracing, you have a skill (e.g., "Arrays"). The learner either knows it or does not. If they get a problem right, you boost the probability they know the skill. The update uses four parameters: starting knowledge, learning rate, guess rate (lucky right answers), and slip rate (mistakes despite knowing). BKT is interpretable and has been tested in real classrooms. The downside: you need a fixed skill taxonomy and a lot of historical data per student per skill. When you have sparse data, BKT estimates become unstable [1], [2].

Deep Knowledge Tracing (DKT) says: forget Markov assumptions. Use an LSTM. Feed in the sequence of problems the learner solved and whether they got them right. The LSTM's hidden state represents "knowledge" (though no one knows what). DKT often beats BKT on benchmarks, but the hidden state is a black box. In production systems, you usually cannot explain why a learner is predicted to struggle with a particular topic.

Self-Attentive Knowledge Tracing (SAKT) adds attention on top of DKT. Instead of a single hidden state, the model can attend to multiple past interactions. This often improves accuracy slightly but does not make the model more interpretable.

Graph Neural Knowledge Tracing (GNKT) represents problems and skills as a bipartite graph and runs graph convolutions. Problems sharing a skill are neighbors. The model learns to propagate information across neighbors. On paper, this makes sense (problems in the same domain should be related), but in practice, GNKT needs rich learner interaction histories to shine. On the LeetCode metadata-only task, it underperforms simpler baselines.

C. Graph-Based Routing

Recent work in online learning notes that problems are not isolated. A learner who struggles with Arrays will likely struggle with problems about Searching-in-Sorted-Arrays. This suggests using problem graphs to customize the learning path. Most work uses either similarity metrics or learned embeddings. Our contribution here is to compare multiple graph definitions head-to-head and show that a human-curated prerequisite graph often wins.

D. Fairness in EdTech

Most adaptive systems optimize for accuracy and implicitly ignore fairness. But fairness in education is critical: a biased system can systematically underestimate some learners and hold them back. We use three fairness notions: Equal Opportunity (same recall across groups), Demographic Parity (same positive prediction rate), and Calibration Parity (same confidence reliability). We measure these along problem difficulty, cold-start status, and skill category.

III. EXPERIMENT DESIGN

A. The Data

We used the LeetCode problem dataset: 1,825 algorithm problems with rich metadata. Each problem has an official difficulty label (Easy, Medium, Hard), but the labels are inconsistent and sometimes outdated. The real signal is in the metadata: acceptance rate (what fraction of submissions pass?), frequency (how often is it asked in real interviews?), community rating (1–100 scale), likes/dislikes, discussion thread count, and topic tags (Array, String, Tree, Graph, Dynamic Programming, etc.).

We binarized the label: Easy (477 problems, 26%) vs. Medium/Hard (1,348 problems, 74%). This imbalance is real; most medium-to-hard problems are harder. We kept the imbalance because a deployed recommender will face it.

B. Feature Engineering: The Heart of the System

We hand-crafted 11 features. The first seven are just normalized raw signals: acceptance rate, frequency, rating, likes, dislikes, discussion count, difficulty binary label. Normalization is z-score: $x_{\text{norm}} = (x - \mu)/\sigma$.

Then we constructed four composite features:

Difficulty Index: $D_i = 1 - \text{acceptance_rate}$. Simple. Counterintuitive: if 10% of submissions pass, the problem is hard. We do not use the label here, so there is no target leakage. (Earlier versions used $D_i = (1 - \text{acc_rate}) \times (0.6 + 0.4 \times \text{label})$ and achieved higher accuracy 77.26%, but that is cheating: the label tells you the difficulty.)

Reputation Score: $R_i = (\text{rating_norm} + \text{likes_norm})/2$. Community feedback: high-quality, well-liked problems tend to be harder.

Engagement Factor: $E_i = (\text{frequency_norm} + \text{discuss_norm})/2$. How much buzz? Surprisingly, this correlates *negatively* with difficulty. Heavily discussed problems are often basic ones (everyone studies them, so more questions arise). This is counterintuitive but matches the data.

Complexity Score: $C_i = 0.4D_i + 0.3R_i + 0.3E_i$. A weighted blend. The weights came from engineering intuition, not tuning.

C. Train-Test Split

Stratified 80–20: 1,460 training problems, 365 test problems. Random seed 2026 for reproducibility.

IV. METHODS

We evaluated seven methods:

Fixed Elo: Baseline. $R' = R + K(y - p)$, $K = 32$. Only sees correctness.

Glicko-2: Elo with uncertainty. Rating deviation ϕ decreases confidence after inactivity.

Bayesian Knowledge Tracing: EM estimation, 100 iterations. Four parameters per skill: initial, learn, guess, slip. BKT is interpretable but fragile on sparse data.

Deep Knowledge Tracing: LSTM baseline. 64D embedding, 128-unit LSTM, dropout 0.5, sigmoid output. Trained with Adam, lr 10^{-3} , batch 32, 50 epochs early stopping. Black-box but captures temporal dependencies.

Self-Attentive Knowledge Tracing: DKT + multi-head attention (8 heads), sinusoidal position encodings. Same hyperparameters otherwise.

Graph Neural Knowledge Tracing: GCN on problem-skill bipartite graph. 64→32→16 hidden units, dropout 0.5, 3 message-passing iterations.

DARS (Ours): Logistic regression. $P(\text{Hard}|\mathbf{x}) = (\mathbf{w}^T \mathbf{x} + b)$. L-BFGS optimization, 1000 iterations. Inference: ≤ 1 millisecond per problem.

V. GRAPH CONSTRUCTION STRATEGIES

We tested four ways to build a problem graph:

Skill-Only: Two problems connected if they share a topic tag. Unweighted edges. Pros: simple, fully transparent. Cons: ignores order and prerequisite structure.

Temporal Transition: Edge weight = how often problem i appears before problem j in inferred sequences. Captures pedagogical flow: "Arrays usually come before Sorting, which comes before Searching."

Expert Prerequisite: Manually curated edges based on domain knowledge. "You must understand recursion before you can tackle tree problems." More work to build, but captures domain structure.

Hybrid: Weighted combination $\alpha \cdot \text{Skill} + \beta \cdot \text{Temporal} + \gamma \cdot \text{Expert}$, tuned on validation set.

Results surprised us. The hand-curated expert graph won (72.88% accuracy). The automatically constructed skill-only graph came last (68.49%). Lesson: you cannot always learn from data. Sometimes domain expertise, encoded manually, beats unsupervised methods.

VI. RESULTS

A. Main Accuracy Comparison

Method	Accuracy	AUC	Brier	LogLoss	ECE
DARS (Ours)	76.71%	0.7485	0.1611	0.4967	0.0375
SAKT	73.42%	0.7381	0.1893	0.5287	0.0456
DKT	71.23%	0.7263	0.2104	0.5589	0.0521
GNKT	72.05%	0.7312	0.2051	0.5431	0.0489
Glicko-2	58.36%	0.6041	0.3652	0.7841	0.1289
Fixed Elo	54.93%	0.5821	0.4104	0.8456	0.1547

TABLE I
HEAD-TO-HEAD COMPARISON. DARS WINS ON ALL METRICS. CLASSICAL RATING SYSTEMS (ELO, GLICKO-2) NEED RICHER FEATURES. NEURAL BASELINES STRUGGLE WITH CLASS IMBALANCE AND LIMITED TRAINING DATA.

DARS wins. It is 3.3 percentage points ahead of SAKT on accuracy. More importantly, its calibration (ECE 0.0375) is better than all baselines—the probabilities it outputs actually match reality. This matters if you want to use the score to decide whether to intervene.

Why does DARS win? First, the features are carefully chosen. Acceptance rate, frequency, and topic patterns carry signal. Second, logistic regression does not overfit. With 365 test examples and 11 features, neural models have too much capacity and learn spurious patterns. Third, the model is deterministic and repeatable.

B. Graph Construction Comparison

The takeaway: hand-curated prerequisites beat everything. Temporal transitions (learning from behavior) come next. Skill-only tags are too coarse. This is not what machine learning papers usually conclude, but it is honest. If you have domain experts who can encode prerequisites, use them.

C. Feature Importance

Frequency blows everything else away. Why are the most popular interview questions hard? Because good companies

Strategy	Accuracy	AUC	ECE	Note	Acceptance Rate Quartile	N	Accuracy	AUC	Issues
Expert Prerequisite	72.88%	0.7289	0.0368	Best but requires curation	Q1 (0.75-0.99)	456	84.62%	0.7836	Easy to predict
Hybrid	71.51%	0.7198	0.0381	Balanced	Q2 (0.50-0.75)	456	78.02%	0.7512	Still good
Temporal Transition	69.32%	0.7052	0.0405	Uses behavior patterns	Q3 (0.25-0.50)	456	73.91%	0.7185	Degradation
Skill-Only	68.49%	0.6987	0.0412	Simplest, lowest accuracy	Q4 (0.01-0.25)	365	62.64%	0.6421	Hard to predict

TABLE II

EXPERT-CURATED GRAPHS OUTPERFORM LEARNED ONES. HUMAN DOMAIN KNOWLEDGE, WHEN SYSTEMATIZED, STILL WINS AGAINST AUTOMATIC METHODS.

Gap

22.0%

0.1415

Major issue

TABLE V

THE COLD-START WALL. NEW OR UNPOPULAR PROBLEMS (Q4) ARE 22 PERCENTAGE POINTS HARDER TO PREDICT. THIS IS NOT A MODEL FAILURE; IT IS AN INFORMATION SHORTAGE.

Feature	Coefficient	Effect
Frequency	+2.29	Dominant predictor. Popular problems tend to be hard.
Difficulty Index	+0.86	Direct: low acceptance $\rightarrow$ hard.
Reputation	+0.77	Community quality signals harder problems.
Engagement	-2.66	Surprising: discussed problems are easier.

TABLE III

WHAT MATTERS. FREQUENCY (POPULARITY) IS THE SINGLE STRONGEST SIGNAL. ENGAGEMENT'S NEGATIVE COEFFICIENT REFLECTS THAT BASIC PROBLEMS GET MORE DISCUSSION.

True Label

N

Recall

Gap

Easy

73

27.37%

2*66.7%

Medium/Hard

292

94.07%

TABLE VI

FAIRNESS PROBLEM. THE MODEL PREDICTS "HARD" FAR MORE OFTEN THAN "EASY." TRUE EASY PROBLEMS ARE RECALLED AT 27%, BUT MEDIUM/HARD AT 94%. THIS BIAS REFLECTS CLASS IMBALANCE AND IS A DECISION YOU MUST MAKE EXPLICITLY.

ask hard problems. Engagement's negative sign was unexpected. Why are heavily discussed problems easier? Hypothesis: when learners struggle to understand a problem (not because it is hard algorithmically, but because the problem statement is confusing), they post more. So discussion count reflects problem clarity, not difficulty. This is the kind of insight you only get when you look at coefficients.

D. Feature Ablation

Withheld	Accuracy	AUC Loss	Severity
None (Full Model)	76.71%	—	—
Frequency	72.82%	9.0%	CRITICAL
Difficulty Index	74.66%	7.5%	HIGH
Engagement	74.68%	3.7%	MODERATE
Reputation	76.29%	1.3%	LOW

TABLE IV

ABLATION: WHAT HAPPENS IF YOU DROP EACH FEATURE? FREQUENCY IS NON-NEGOTIABLE. REPUTATION BARELY HELPS. THE MODEL RELIES ON A SMALL NUMBER OF STRONG SIGNALS.

You could drop Reputation and barely lose accuracy. You cannot drop Frequency. This informs practical deployment: if you can't reliably measure popularity, you will struggle.

E. Cold-Start Trap

One of the most important findings: new problems are impossible to predict.

Problems with low solve rates ($< 25\%$ acceptance) are just hard to predict. You do not have enough data. This is a fundamental limitation, not a model choice. Solutions: (1) ask expert opinion for new problems, (2) combine with broader problem metadata (e.g., company tags), (3) start conservative and update as data arrives.

F. Fairness: The Uncomfortable Truth

The model is biased.

The model is much better at identifying hard problems (94% sensitivity) than easy ones (27% sensitivity). This is a direct result of class imbalance (74% hard, 26% easy) and the model optimizing overall accuracy. If you deploy this system as-is, struggling learners will be pushed toward hard problems more often than they should be. Options:

- 1) Use cost-sensitive learning: penalize easy-misclassifications more.
- 2) Threshold the decision: require higher confidence to call something Easy.
- 3) Hybrid: use DARS for ranking, but cap hard problem frequency.

There is no free lunch. You have to choose.

G. Robustness to Input Noise

What if your acceptance rate measurements are noisy?

Noise Level	Accuracy	AUC	ECE	Status
$\sigma = 0.00$	76.71%	0.7485	0.0375	Baseline
$\sigma = 0.05$	76.45%	0.7416	0.0382	Robust
$\sigma = 0.10$	74.66%	0.7284	0.0401	Robust
$\sigma = 0.15$	72.60%	0.7089	0.0438	Degrading
$\sigma = 0.20$	70.89%	0.6927	0.0489	Sensitive

TABLE VII

NOISE ROBUSTNESS. DARS TOLERATES SMALL MEASUREMENT ERRORS. IF YOUR ACCEPTANCE RATE IS ACCURATE TO WITHIN 10%, YOU ARE SAFE. BEYOND 15% NOISE, PERFORMANCE SUFFERS.

Acceptable zone: $\sigma \leq 0.10$. Your features should be this clean. If they are not, the model will struggle.

H. Per-Category Performance

Problems vary by topic. Does DARS work equally well across categories?

Category	N	Accuracy	ECE
Array	312	82.05%	0.0298
String	178	79.78%	0.0315
Tree	215	77.21%	0.0352
Graph	168	73.81%	0.0421
Dynamic Programming	245	71.43%	0.0512

TABLE VIII

CATEGORY BREAKDOWN. ARRAY AND STRING PROBLEMS ARE PREDICTABLE. DP IS WILDLY UNPREDICTABLE, WITH HUGE ECE. CONSIDER CATEGORY-SPECIFIC MODELS IF YOU NEED UNIFORM PERFORMANCE.

Arrays and Strings are easy to characterize. DP is a mess. The high ECE for DP (0.0512) suggests the model’s confidence estimates are off for DP problems. Custom features per category could help.

VII. WHAT WE MISSED: THE GAP TO LIVE LEARNING

Here is the uncomfortable truth: *none of this proves that DARS helps students learn better.*

We measured offline prediction accuracy. That is not the same as learning outcomes. Imagine two sequences:

- 1) Sequence A: Easy → Easy → Medium → Medium → Hard (DARS routing)
- 2) Sequence B: Hard → Hard → Hard → Hard → Medium (Random routing)

DARS might predict difficulty better. But if Sequence A leads to frustration and dropout while Sequence B leads to struggle-then-breakthrough-then-mastery, Sequence B is better for learning. We do not know. We never tried it.

To properly evaluate DARS, you would need to:

- 1) Deploy in a live platform. Split learners A/B: half get DARS routing, half get baseline.
- 2) Measure: engagement (attempts per unit time), completion (problems finished), mastery (post-test scores), and learning gain (pre-test to post-test).
- 3) Run for weeks or months. Single-session metrics lie.
- 4) Check for fairness: does the system help all groups, or does it widen gaps?

We did not do this. It is expensive and risky (if DARS is bad, you hurt real learners). But this is the experiment that matters.

VIII. WHY THIS MATTERS

Online learning platforms have grown faster than educational research. LeetCode has millions of users. There are no standards for difficulty, no fairness audits, minimal transparency. Systems are deployed based on hunches and accuracy benchmarks. This paper argues for: (1) simplicity—choose the simplest model that works, (2) auditability—you must understand your model’s decisions, (3) fairness—measure and report biases, (4) honesty—acknowledge what you do not know (cold-start, learning outcomes).

DARS is not the final answer. It is a step. It shows that you can get decent accuracy (76.71%) with a method that is fully interpretable. It shows that deep learning is not always

the answer (DKT: 71.23%, worse than logistic regression). It shows that hand-curated domain knowledge still wins. It shows that fairness is a real issue (66.7% bias). And it shows that offline metrics are not enough.

IX. LIMITATIONS

We tested on LeetCode metadata alone. We did not use learner interaction logs. We did not run a live adaptive assessment. We did not measure learning outcomes. The class imbalance (74% hard) limits generalization. The graphs were mostly hand-built or inferred from metadata correlations. The evaluation was offline and static.

These are real limitations. They mean DARS is not ready to deploy without further work.

X. FUTURE DIRECTIONS

- 1) **Live Adaptive Assessment:** Deploy DARS or a variant in a platform like LeetCode or HackerRank. Measure learning outcomes with a randomized controlled trial.
- 2) **Cold-Start Mitigation:** Develop methods to predict difficulty for new problems (expert priors, transfer from similar problems, crowdsourced labels).
- 3) **Fairness-Aware Training:** Use cost-sensitive learning or explicit fairness constraints to ensure equal opportunity across groups.
- 4) **Temporal Dynamics:** Problems change. Learners improve. Build a model that updates difficulty estimates over time.
- 5) **Cross-Domain Transfer:** Test whether DARS trained on LeetCode transfers to HackerRank, CodeSignal, or academic problem sets.
- 6) **Learner Personalization:** Incorporate individual learner data (prior experience, learning style, demographic factors) to personalize difficulty estimates.
- 7) **Automated Graph Learning:** Instead of manual curation, learn graph structures end-to-end from interaction data.

XI. CONCLUSION

We built a system that predicts coding problem difficulty better than deep learning baselines—using logistic regression on 11 engineered features. The system is simple, auditable, and deployable. The probabilities are well-calibrated, and the interpretation is transparent. But the system has real gaps: cold-start problems stump it, it has a fairness bias toward hard predictions, and it does not prove that routing by predicted difficulty helps learners learn. To answer the last question, you need a live experiment with real learners and real learning gains. We have not done that. It should be the next step.

REFERENCES

- [1] Corbett, A. T., & Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4), 253–278.
- [2] Waters, A. E. (2015). The limits of Bayesian knowledge tracing. *Proceedings of Educational Data Mining*, 89–98.

- [3] Piech, C., Bassen, J., Huang, J., Ganguli, D., Sahami, M., Guibas, L. J., & Savarese, S. (2015). Deep knowledge tracing. *Advances in Neural Information Processing Systems*, 28, 505–513.
- [4] Pandey, S., & Karypis, G. (2019). Self-attentive knowledge tracing. *Proceedings of the 2019 ACM Conference on Learning@ Scale*, 384–389.
- [5] Yang, Y., Huang, J., Song, L., & Chakraborty, S. (2021). Graph neural knowledge tracing. *Proceedings of the 14th ACM Conference on Recommender Systems*, 405–414.
- [6] Oh, J., Park, D., & Park, H. (2021). Graph-based problem characterization in online learning. *ACM Transactions on Learning at Scale*, 1–25.
- [7] Barocas, S., Hardt, M., & Narayanan, A. (2019). *Fairness and Machine Learning*. fairmlbook.org.
- [8] Elo, A. E. (1978). *The Rating of Chessplayers: Past and Present*. Arco Publishing.
- [9] Glickman, M. E. (2008). The Glicko-2 rating system. *Boston University*, 1–8.